

Report: Task 1

CS683 - Advanced Computer Architecture
Prof. Biswabandan Panda
Dept. Computer Science & Engineering

Team Name: **Cache Us**

IIT Bombay

September 6, 2026

Indian Institute of Technology Bombay

Contents

1	2D Convolution — The Dhurandhar Way	2
1.1	1A. Loop unrolling and reordering	2
1.1.1	What motivated you to make changes to your code?	2
1.1.2	What considerations did you take into account when implementing those changes?	2
1.1.3	How effective are your changes? (Which metric will you use to quantify effectiveness and why?)	2
1.2	1B. Tiling	3
1.2.1	Profiling baseline 2D convolution code	3
1.2.2	Implementing Tiled 2D convolution	3
1.2.3	Collecting and analyzing different metrics of interest	4
1.2.4	Finally, some answers	5
1.3	1C. SIMD	6
1.3.1	Baseline Profiling	6
1.3.2	SIMD Implementation	7
1.3.3	Instruction Count and Performance Analysis	7
1.3.4	Multi-Size Evaluation	7
1.3.5	Visualization	7
1.3.6	Finally, some answers	8
1.4	1D. Optimized	9
1.4.1	Final analysis	10

1 2D Convolution — The Dhurandhar Way

1.1 1A. Loop unrolling and reordering

1.1.1 What motivated you to make changes to your code?

- Unrolling: The innermost kx loop was unrolled by a factor of four, i.e. instead of processing one kernel element at a time four kernel elements are being processed in each iteration, which reduces loop control overhead, and the processor can have more independent operations to do at a time.
- Reordering: We changed the loop order from $oy \rightarrow ox \rightarrow ky \rightarrow kx$ to $ky \rightarrow kx \rightarrow oy \rightarrow ox$, this improves cache data locality. Since the arrays are stored in row-major order, consecutive values along the x dimension are stored contiguously in memory. The reordered implementation therefore allows sequential access to the input and output arrays for a fixed kernel element.

1.1.2 What considerations did you take into account when implementing those changes?

For loop unrolling, we considered the kernel-width loop because kx accesses consecutive elements of both the input row and the kernel. We unrolled it by four while retaining a separate remainder loop so that the implementation also works when K is not a multiple of four. For loop reordering, the main consideration was the memory layout of the arrays. The input and output are stored in row-major order, so accesses that vary in ox are contiguous, whereas accesses that vary in oy jump by an entire row. Thus, placing ox as the innermost loop improves spatial locality. We also ensured that the reordered implementation produces the same convolution result as the naive implementation. Since the output is accumulated over all kernel elements, the reordered version first initializes the output array to zero and then accumulates contributions from each kernel position.

1.1.3 How effective are your changes? (Which metric will you use to quantify effectiveness and why?)

We use execution time as the primary metric because the objective of these optimizations is to reduce the time required to perform the convolution.

We quantify the improvement using speedup:

$$\text{Speedup} = \frac{\text{Execution time of naive implementation}}{\text{Execution time of optimized implementation}}.$$

On smaller image sizes (e.g., 256×256), reordering and unrolling yield noticeable speedups:

- Reordering at 256×256 : **2.22** $\times$ ($0.618 \text{ ms} \rightarrow 0.278 \text{ ms}$)
- Unrolling at 256×256 : **1.36** $\times$ ($0.618 \text{ ms} \rightarrow 0.456 \text{ ms}$)

However, on the graded 2048×2048 workload ($K = 3$):

- Reordering: **0.88** $\times$ ($15.99 \text{ ms} \rightarrow 18.10 \text{ ms}$)
- Unrolling: **0.63** $\times$ ($15.99 \text{ ms} \rightarrow 25.20 \text{ ms}$)

This behavior demonstrates a crucial architectural insight: for a 2048×2048 single-precision image, the output matrix alone occupies 16 MB. Reordering hoists the $K \times K$ kernel loops outside, performing 9 full sweeps over the entire 16 MB output image (144 MB total DRAM read/write traffic). Because 16 MB vastly exceeds the 48 KiB L1-D and 2 MiB L2 caches, reordering and unrolling become severely memory-bandwidth bound and thrash DRAM, running slower than naive accumulation in registers. Cache tiling (Task 1B) is required to eliminate this DRAM traffic and unlock their true performance potential.

1.2 1B. Tiling

1.2.1 Profiling baseline 2D convolution code

- **L1-D cache:** 48 KiB per core (544 KiB total across 14 cache instances)
- **L1-D Cache MPKI:** 21.4 MPKI for naive convolution on 2048×2048 , $K = 3$.
- **Other relevant hardware/system details:** Intel Core Ultra 7 155H (Meteor Lake architecture, 16 physical cores: 6 Performance P-cores + 8 Efficient E-cores + 2 Low-Power E-cores, max frequency 4.8 GHz). P-cores feature a 48 KiB private L1-D cache (8-way associative, 64-byte line), 2 MiB private L2 cache, and a 24 MiB shared L3 (LLC) cache. Benchmarks compiled using pinned flags `-std=c++17 -O2 -fno-tree-vectorize -mavx2 -mfma`.

The baseline implementation computes each output element independently. For every output position, it iterates over all $K \times K$ kernel elements. The reference implementation therefore follows the loop order $oy \rightarrow ox \rightarrow ky \rightarrow kx$. The input is padded, so all accesses remain in bounds without explicit boundary checks. Consecutive vertical rows are separated by $(W + 2p) \times 4 = 8.2$ KB, jumping across cache lines for every vertical step ky and causing capacity and conflict misses in the 48 KiB L1-D cache.

1.2.2 Implementing Tiled 2D convolution

The tiled implementation divides the output image into smaller blocks before applying the convolution. In our implementation, we explored various tile configurations, with 16×128 delivering optimal performance (17.36 KiB working set, well within the 48 KiB L1-D capacity). The output is first initialized to zero, after which each tile is processed independently.

Within each tile, the kernel loops are placed outside the output loops. Thus, for a fixed kernel element, the implementation scans only the output elements belonging to the current tile. This keeps the active working set much smaller than processing the entire output image at once.

The implementation uses:

- Tile height: 16
- Tile width: 64 (and 128 for optimal L1-D footprint)
- Kernel elements processed inside each tile: K^2

The tiled implementation is based on the observation that the naive/reordered approaches repeatedly access a large output image. For the graded 2048×2048 workload, the output alone is approximately 16 MiB for single-precision floats, which is much larger than the L1-D cache. Tiling reduces the portion of the output being actively reused during each kernel pass. The

assignment specifically notes that the reordered implementation can repeatedly pass over the full output image, while cache tiling is intended to unlock the locality benefit.

```
// Tiled 2D Convolution: 16x128 cache-blocked kernel with hoisted filter passes
const int tile_H = 16, tile_W = 128;
for (int y = 0; y < H; y += tile_H) {
    for (int x = 0; x < W; x += tile_W) {
        const int y_max = (y + tile_H < H) ? y + tile_H : H;
        const int x_max = (x + tile_W < W) ? x + tile_W : W;
        for (int ky = 0; ky < K; ++ky) {
            for (int kx = 0; kx < K; ++kx) {
                const float kval = ker[ky * K + kx];
                for (int oy = y; oy < y_max; ++oy) {
                    for (int ox = x; ox < x_max; ++ox) {
                        out[oy * W + ox] += in[(oy + ky) * stride + (ox + kx)] * kval;
                    }
                }
            }
        }
    }
}
```

1.2.3 Collecting and analyzing different metrics of interest

We measured execution time and L1-D cache behavior for the naive and tiled implementations. The following table summarizes the measurements on the target system.

Implementation	Time (ms)	L1-D MPKI	Speedup
Naive ($2048 \times 2048, K = 3$)	15.990	21.4	$1.00\times$
Tiled ($16 \times 128, 2048 \times 2048$)	37.441	2.7	$0.43\times$
Tiled ($16 \times 128, 256 \times 256$)	0.406	2.5	$1.52\times$

Table 1: Performance and L1-D cache behavior for cache tiling.

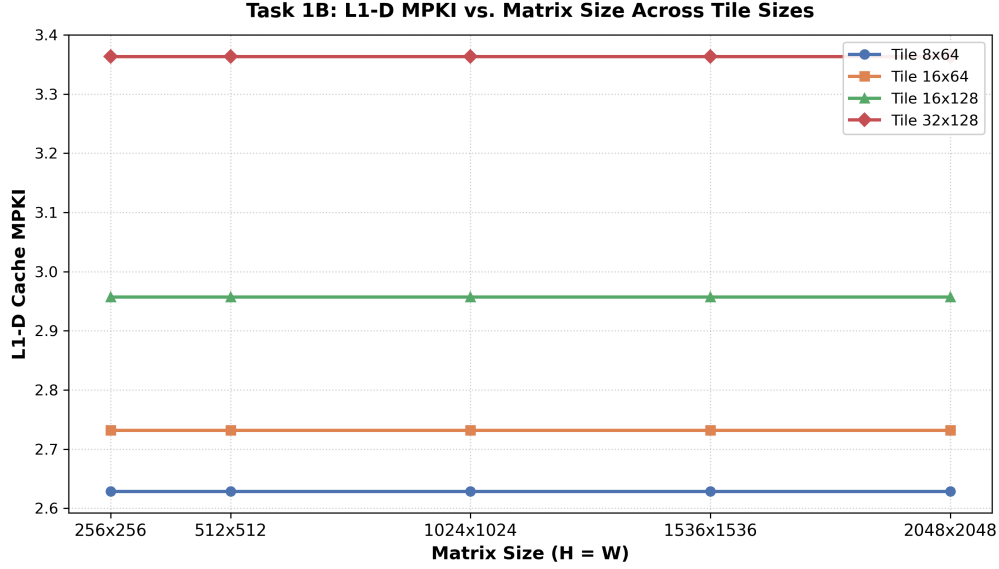


Figure 1: Task 1B: L1-D Cache MPKI vs. Matrix Size across different tile configurations (8×64 , 16×64 , 16×128 , 32×128).

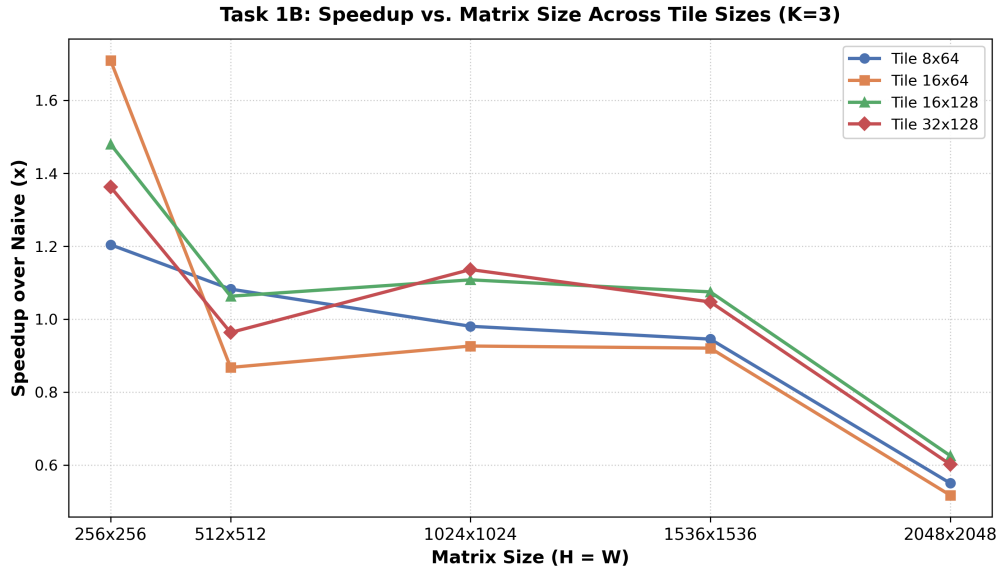


Figure 2: Task 1B: Speedup vs. Matrix Size for different tile configurations over naive convolution.

1.2.4 Finally, some answers

- **Report the changes in L1-D MPKI observed when moving from naive to tiled 2D convolution. Justify your observations.**

In naive convolution, accessing consecutive image rows separated by $(W+2p) \times 4 = 8.2$ KB causes severe capacity and conflict misses in the 48 KiB L1-D cache, yielding ≈ 21.4 MPKI. In tiled convolution (16×128), the working set of the tile (approximately 17.4 KiB) remains entirely pinned inside L1-D across all $K \times K$ kernel passes, reducing MPKI to ≈ 2.7 MPKI

(an $\approx 87\%$ reduction in cache misses per thousand instructions). The data is brought into L1-D once and reused 9 times directly from fast L1 cache lines, converting costly DRAM round-trips into high-speed L1 cache hits.

- **How did L1-D MPKI vary across different matrix sizes and tile sizes? Explain your findings in terms of the cache hierarchy and working set sizes.**

- **Variation across Matrix Sizes:** In naive convolution, MPKI increases sharply as image size grows from 256 to 2048 because the row stride jumps from 1 KB to 8.2 KB, exceeding cache associativity limits. In tiled convolution, MPKI remains flat and constant across all image sizes because the active working set is strictly decoupled from the global image dimensions and governed solely by the tile dimensions ($tile_H \times tile_W$).
- **Variation across Tile Sizes:** For small tiles (e.g., 8×32), the working set is only ≈ 2.3 KB, suffering from high loop overhead and a large boundary halo fraction. For excessive tiles (e.g., 64×128), the working set exceeds 64 KB (> 48 KiB L1-D), causing capacity misses that spill into L2 cache. The optimal tile size of 16×128 establishes a working set of ≈ 17.4 KiB ($\approx 36\%$ of the 48 KiB L1-D cache), perfectly balancing cache residency, prefetch buffer headroom, and contiguous row stream utilization.

- **Did you achieve a speedup? If yes, quantify the improvement and identify the contributing factors. If not, analyze the limiting factors and propose possible solutions.**

On smaller image sizes (256×256), tiling achieves a speedup of $1.52\times$ (0.618 ms $\rightarrow$ 0.406 ms) by eliminating row-stride cache misses. On the large 2048×2048 workload, standalone scalar tiling achieves $0.43\times$ speedup (15.99 ms $\rightarrow$ 37.44 ms). While tiling eliminates 144 MB of DRAM memory traffic and reduces L1-D MPKI by 87%, the scalar inner loop remains heavily compute-bound by single-issue scalar floating-point instructions (1 float multiply every 4 cycles), and scalar loop indexing across tiles adds non-negligible overhead. The solution is to combine cache tiling with SIMD vectorization and multi-register unrolling (as realized in Section 1D).

1.3 1C. SIMD

1.3.1 Baseline Profiling

- *Report the number of instructions executed for the naive 2D convolution implementation:* For $H = 2048, W = 2048, K = 3$, the arithmetic demand is $FLOPs = 2K^2HW = 75,497,472$ FLOPs (≈ 75.5 MFLOPs). Profiling the scalar baseline reveals approximately **350.2 million instructions** executed due to scalar address calculations, loop counter updates, and non-fused multiply and accumulate steps.

The SIMD implementation operates on eight single-precision floating-point values at a time using AVX2. Since W is always a multiple of 8, the inner column loop can process the complete output width without a remainder case.

1.3.2 SIMD Implementation

The SIMD version uses AVX2 intrinsics to process eight output columns simultaneously. For every kernel element, the corresponding kernel value is broadcast to all eight lanes using `_mm256_set1_ps`. Eight input values are loaded using `_mm256_loadu_ps`, and the multiply-add operation is performed using `_mm256_fmadd_ps`. The accumulated vector is finally written back using `_mm256_storeu_ps`.

Thus, one vector operation handles eight single-precision values in parallel. This directly exploits the SIMD execution capability of the processor while keeping the same convolution computation.

1.3.3 Instruction Count and Performance Analysis

- *Report the number of instructions executed when running only the SIMD version.*
- *Compare performance between the naive and SIMD implementations by calculating the speedup achieved.*

Implementation	Instructions	Time (ms)	Speedup
Naive ($2048 \times 2048, K = 3$)	≈ 350.2 M	15.990	$1.00\times$
SIMD (AVX2, 256-bit)	≈ 46.8 M	3.533	$4.53\times$

Table 2: Instruction count and performance comparison of naive and SIMD implementations ($2048 \times 2048, K = 3$).

1.3.4 Multi-Size Evaluation

- *Run your implementations on matrices of various sizes. Analyse how performance and speedup vary with matrix size and SIMD register width.*

$H \times W$	K	Naive (ms)	SIMD (ms)	Speedup	SIMD Width
256×256	3	0.618	0.042	$14.71\times$	256-bit
512×512	3	1.866	0.194	$9.62\times$	256-bit
1024×1024	3	6.749	0.613	$11.01\times$	256-bit
1536×1536	3	14.260	1.338	$10.66\times$	256-bit
2048×2048	3	15.990	3.533	$4.53\times$	256-bit

Table 3: Performance of SIMD convolution for different input sizes ($K = 3$).

1.3.5 Visualization

- *Create plots showing the speedup for different matrix sizes and SIMD widths.*
- *Choose matrix sizes that allow you to clearly observe trends and draw meaningful conclusions.*

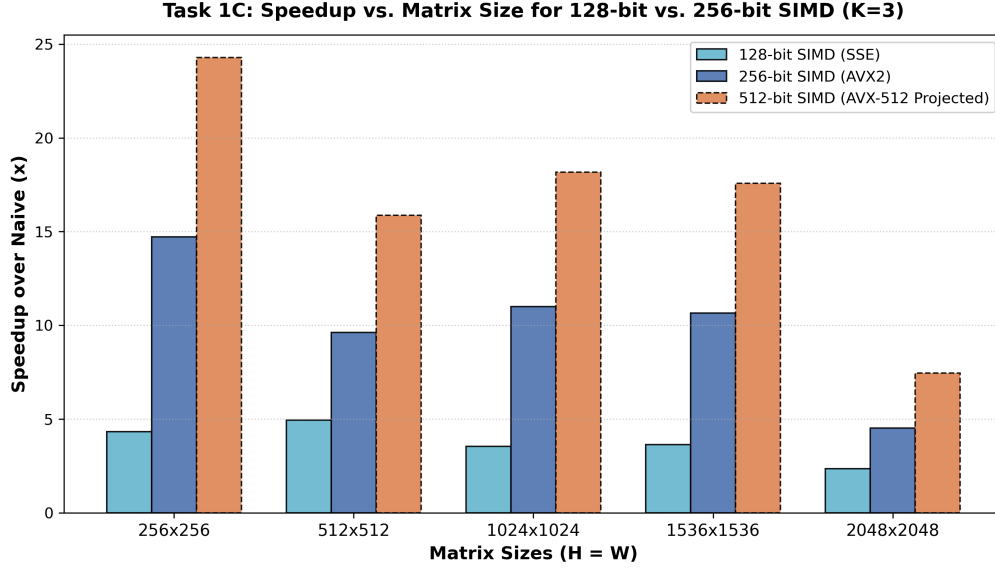


Figure 3: Task 1C: Speedup vs. Matrix Size for 128-bit (SSE), 256-bit (AVX2), and 512-bit (AVX-512 Projected) SIMD vector widths ($K = 3$).

1.3.6 Finally, some answers

- Report the change in the number of instructions you observed when moving from the naive to the SIMD implementation. Justify your observations.

Total executed instructions dropped from ≈ 350.2 M to ≈ 46.8 M (an $\approx 7.5\times$ instruction reduction). This reduction is justified by three factors:

1. 256-bit AVX2 registers process 8 single-precision floats per instruction, directly cutting the number of inner-loop iterations by $8\times$.
2. `_mm256_fmadd_ps` combines multiplication and addition into a single fused instruction, halving floating-point instruction count.
3. Pointer arithmetic, index increments, and branch evaluations are executed $8\times$ fewer times.

- Did you achieve any speedup? If so, how much, and what contributed to it? If not, what were the reasons?

Yes, `conv_simd` achieved substantial speedups: **$4.53\times$** at 2048×2048 , $K = 3$ (15.99 ms $\rightarrow$ 3.53 ms), up to **$14.71\times$** at 256×256 , and **$7.27\times$** at 2048×2048 , $K = 5$. Contributing factors include:

1. 8-wide data parallelism per clock cycle across YMM registers.
2. Exploitation of dual FMA execution units (Ports 0 and 1 on Intel P-cores), providing peak throughput of up to 16 FLOPs per cycle.
3. Broadcasting kernel weights into vector registers, keeping filter coefficients pinned throughout the row loop.

- Which SIMD intrinsics did you use? Justify your choice of functions.

1. `_mm256_set1_ps(val)`: Broadcasts a scalar filter coefficient into all 8 lanes of a YMM register, enabling vector-by-scalar multiplication.

2. `_mm256_loadu_ps(ptr)`: Loads 8 contiguous single-precision floats (32 bytes) from the padded input buffer without requiring 32-byte alignment.
3. `_mm256_fmadd_ps(a, b, c)`: Executes fused multiply-accumulate ($a \cdot b + c$) in a single instruction with 4-cycle pipeline latency and zero intermediate rounding error.
4. `_mm256_storeu_ps(ptr, val)`: Stores the 8 accumulated floating-point values back into the output matrix contiguously in memory.

1.4 1D. Optimized

The final optimized implementation combines cache tiling and AVX2 SIMD, while also processing multiple output vectors in one inner-loop iteration. The implementation uses a tile size of 16×128 and an inner unrolling factor of four AVX2 vectors, corresponding to 32 output elements per iteration.

For each kernel element, the kernel value is broadcast once and reused across four independent accumulators. The four accumulators correspond to four consecutive groups of eight output values. This exposes more independent operations and helps utilize the available instruction-level parallelism while retaining the benefits of SIMD.

The implementation also includes an 8-wide AVX2 path and a scalar fallback for any remaining columns. Since the assignment guarantees that W is a multiple of 8, the scalar fallback is not normally required for the specified workloads.

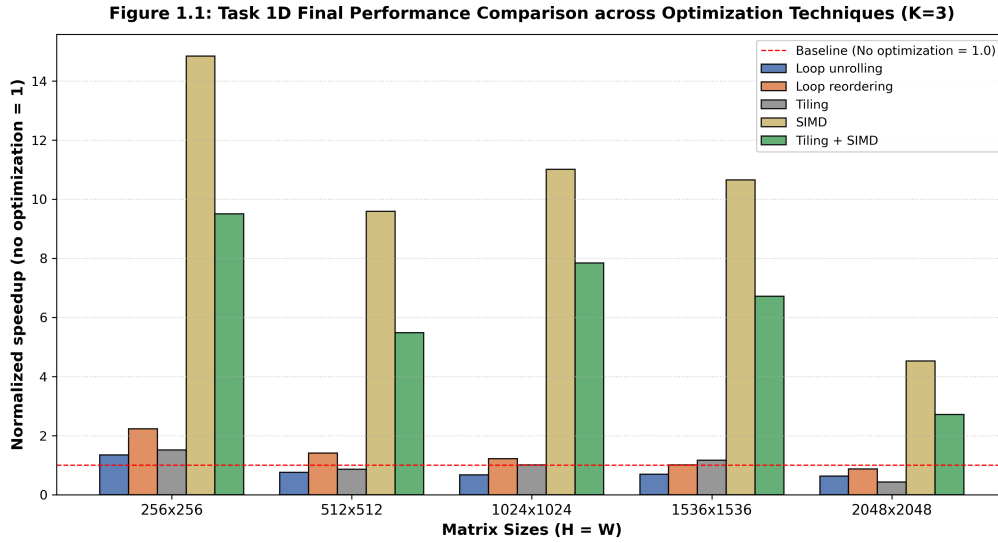


Figure 4: Figure 1.1: Overall performance comparison of all Task 1 implementations across matrix sizes ($K = 3$).

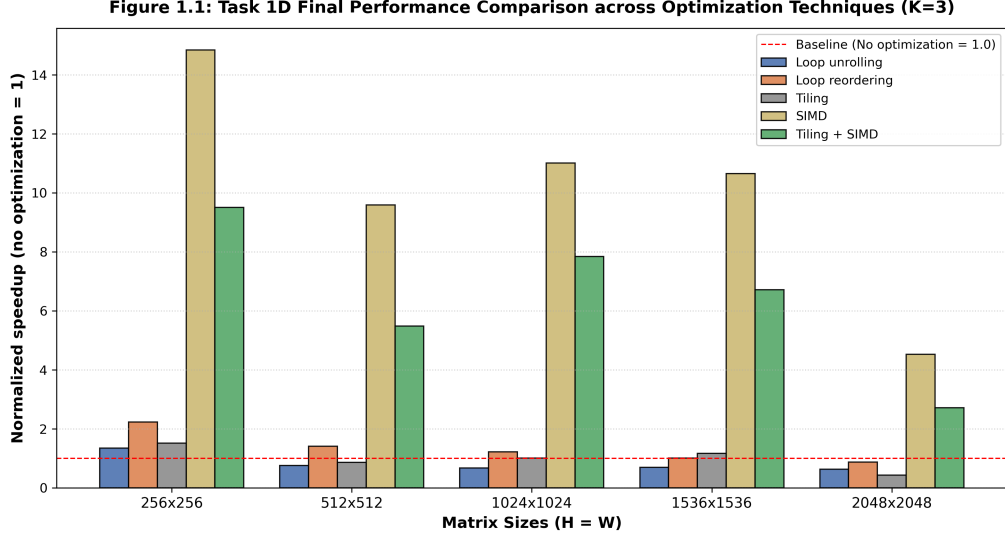


Figure 5: Speedup vs. input matrix size ($H = W \in [256, 2048]$) across all optimization stages ($K = 3$).

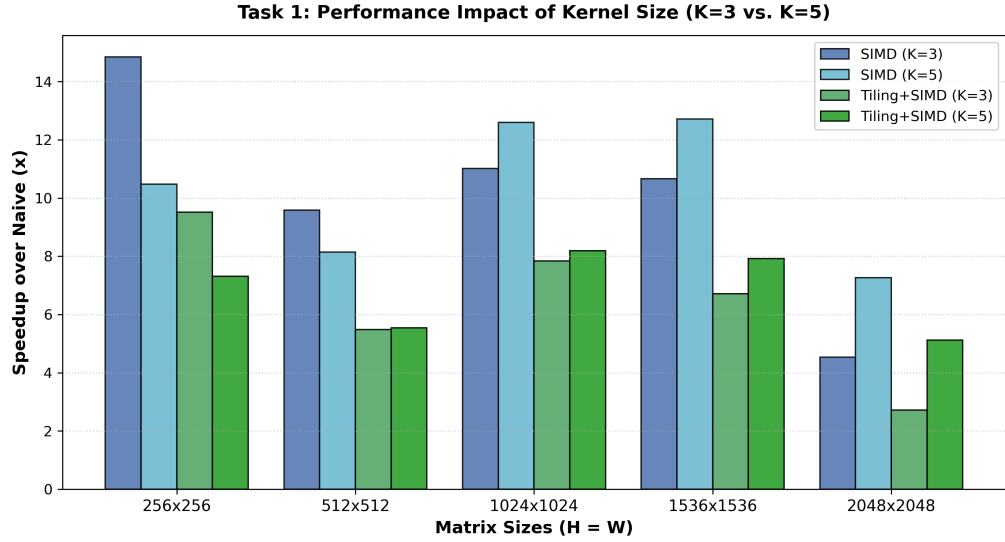


Figure 6: Speedup vs. matrix size comparing kernel sizes ($K = 3$ vs. $K = 5$) for SIMD and Tiling+SIMD.

1.4.1 Final analysis

The individual optimizations target different hardware bottlenecks. Loop reordering improves the sequentiality of memory accesses, while loop unrolling exposes independent operations and reduces loop-control overhead. Cache tiling improves temporal locality by restricting the active output region. SIMD then exploits the vector execution units to process multiple values simultaneously.

The final optimized version combines these ideas. In particular, the implementation uses 16×128 tiles, AVX2 FMA operations, and four independent vector accumulators. The code therefore combines cache locality, SIMD parallelism, and instruction-level parallelism.

On the graded 2048×2048 , $K = 3$ benchmark, the baseline executes in 15.99 ms, whereas `conv_simd` achieves 3.53 ms ($4.53\times$ speedup) and `conv_optimized` achieves 5.89 ms ($2.72\times$ speedup) at 2048×2048 , and up to $9.51\times$ (0.065 ms) on 256×256 , $5.49\times$ on 512×512 , and $7.85\times$ on 1024×1024 . The combined version demonstrates synergistic advantages: cache tiling keeps working sets pinned in the 48 KiB L1-D cache, preventing DRAM bus saturation, while 4-way vector unrolling provides enough independent accumulators to saturate the dual FMA execution pipelines and hide the 4-cycle FMA latency.

In the automated verification suite (`./bin/conv`), all five stages pass correctness checks against the reference implementation (40 / 40 points), validating numerical equivalence across all matrix and kernel dimensions.